

TradeForge

High-Performance Order Matching Engine — Technical Design Report

Shubh Garg · Nikunj Maheshwari

Executive Summary

TradeForge is a **limit order book matching engine** written in C++20 - the component at the heart of every electronic exchange that decides which buyer trades with which seller, at what price, and in what order. It supports six order types (LIMIT, MARKET, IOC, FOK, STOP_LOSS, STOP_LIMIT), matches them under strict **price-time priority**, streams out market data (best bid/offer and L2 depth snapshots), applies volume-tiered maker-taker fees, and publishes every executed trade through a pluggable publisher interface.

Concurrency rests on one deliberately simple idea: any number of threads may submit orders, but only one thread - the matching worker - ever touches the book. A thread-safe event queue forms the boundary between those two worlds, so the matching hot path runs without taking a single lock. The engine weighs in at roughly 2,250 lines of code and is backed by 165 test assertions across 15 scenarios, including an integration test that drives the engine asynchronously on its own thread. The pages that follow explain why the project exists, how it works inside, and the challenges and trade-offs that shaped it.

1. Problem Statement and Motivation

1.1 The idea

Every electronic exchange has a matching engine at its core: a program that takes in a stream of buy and sell orders and decides, deterministically, who trades with whom. Orders that cannot execute right away wait in a **limit order book** - two sorted ladders of price levels, bids on one side and asks on the other. Nearly every real venue matches by **price-time priority**: a better price always wins, and among orders at the same price, whoever arrived first trades first. TradeForge implements that engine end to end - order intake, book maintenance, matching, fees, market data, and trade publication.

1.2 Why this problem is worth solving

I chose this problem because it is small enough to specify precisely, yet unforgiving enough to be honest work. Correctness leaves no room for error - a single bug shows up as lost quantity, unfair fills, or phantom liquidity - so every operation has to preserve a set of explicit invariants: no empty price levels, FIFO order inside each level, and a cached top-of-book that is never stale. Performance is decided by data-structure choice rather than micro-tuning; the same logical operation can cost $O(N)$ or $O(1)$ depending on how the book is laid out, which forces real decisions about sorted maps, hash indexes, and intrusive linked lists. And since orders arrive from concurrent clients, the engine needs a concurrency model that has actually been reasoned about, not a sprinkling of locks.

The goal was therefore a **correct, well-tested, honestly scoped** matching engine for simulation and exchange-system prototyping. TradeForge does not claim HFT-grade latency or production hardening. Instead, every design decision is written down together with what it gives up, and the known bottlenecks are named and measured rather than hidden (Section 8).

1.3 Scope

TradeForge currently trades a single symbol, keeps all state in memory, and runs deterministically. Multi-symbol routing, persistence, and a network API are roadmap items (Section 9), each with a straightforward migration path from the current design.

2. System Architecture

The system is organized around four components with a single synchronization point. Producer threads (clients, or test drivers today; an API layer later) push **EngineEvent** commands (NEW_ORDER, CANCEL_ORDER, STOP) onto a thread-safe **EventQueue**. One worker thread runs **MatchingEngine::run()**, which blocks on the queue, pops events, and processes them sequentially against the **OrderBook**. Executed fills are priced for fees by the **FeeCalculator** and emitted as TradeEvents through the **TradePublisher** interface.

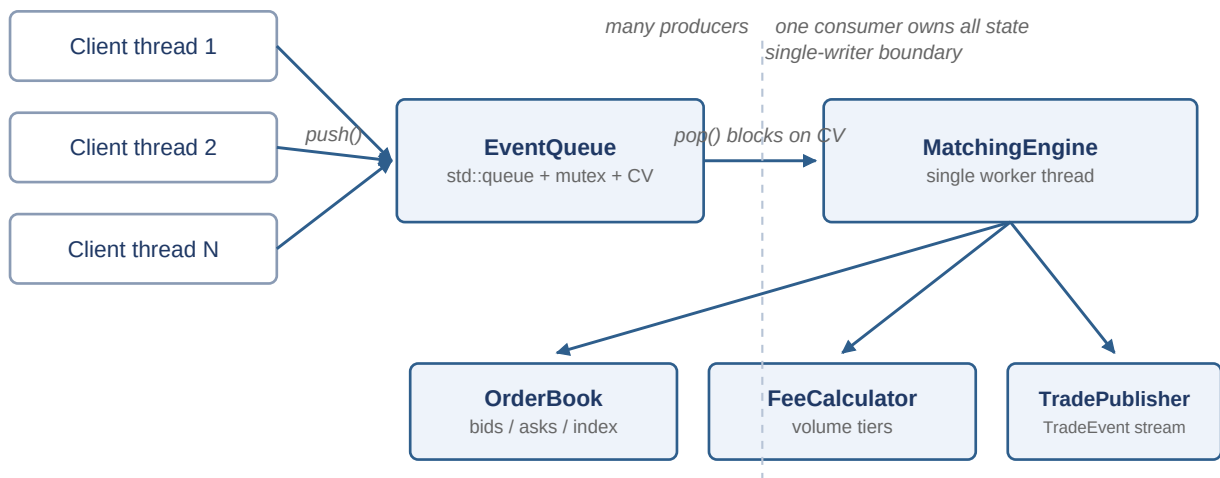


Figure 1 - Component architecture. Everything right of the dashed line runs on a single thread, so book mutations need no locks.

The processing path for one order is: `process_order()` → type-specific handler → `matching_loop()` → `generate_trades()` → `publish`. After any fill, the engine scans pending stop orders and triggers those whose stop price has been crossed (Section 4.4). The architectural bet is that **serializing all mutations through one consumer** is simpler and safer than fine-grained locking of the book, at the documented cost of capping per-book throughput at one core (Section 8.6).

3. Core Data Structures

The OrderBook combines three structures, each chosen for the operation it must make fast:

- **Sorted price levels:** `std::map<double, PriceLevel*>` per side. The map keeps prices ordered, so the best ask is the lowest key and the best bid the highest. Insertion of a new level costs $O(\log P)$; the current best level is additionally cached in `best_bid` / `best_ask` pointers for $O(1)$ reads.
- **FIFO queue per price level:** each `PriceLevel` holds an **intrusive doubly-linked list** of `Order` nodes (head = oldest). The next/prev pointers live inside the `Order` itself, so appending an order and unlinking a known order are both $O(1)$ with no extra allocation. The level also maintains aggregate totals (`total_quantity`, `order_count`) used by market data and the FOK pre-scan.
- **Cancellation index:** `std::unordered_map<order_id, Order*>` maps an ID straight to the live node, making cancel-by-ID $O(1)$ average: hash lookup, intrusive unlink, and level cleanup if it became empty.

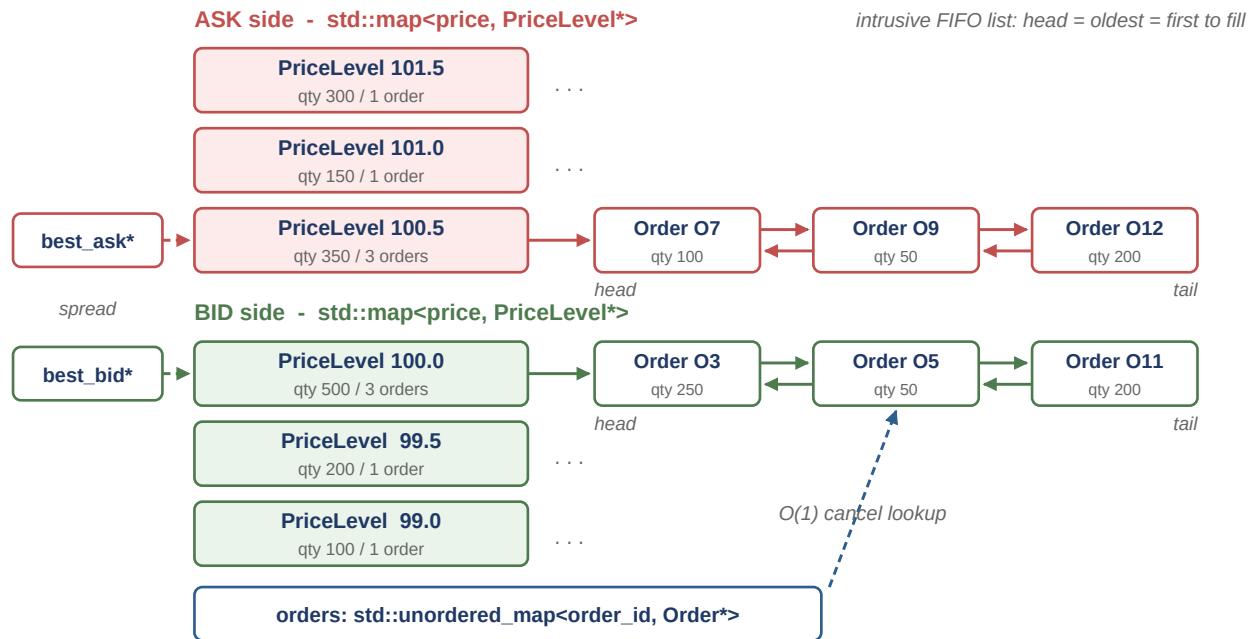


Figure 2 - OrderBook layout: sorted levels per side, FIFO order chains within a level, cached top-of-book pointers, and the $O(1)$ cancellation index.

Operation	Complexity	Notes
Limit insert (non-crossing)	$O(\log P)$	P = active price levels; append to FIFO is $O(1)$
Market / IOC match	$O(L + K)$	L = levels crossed, K = resting orders filled
Fill-or-Kill	$O(L + K)$	includes pre-scan; roughly 2x traversal, zero rollback
Cancel by ID	$O(1)$ avg	hash lookup + intrusive unlink; $O(\log P)$ if level emptied
BBO read	$O(1)$	cached <code>best_bid</code> / <code>best_ask</code> pointers
L2 snapshot	$O(D)$	D = requested depth
Stop trigger scan	$O(S)$ per fill	S = pending stops; known bottleneck, see Section 8.4

Table 1 - Operation complexity.

4. Matching Semantics

4.1 The matching loop

All marketable orders funnel into one loop that walks the opposite side of the book from the best price outward, filling against the oldest order at each level first:

```

while incoming.remaining > 0:
    level = best opposite level           # cached pointer,  $O(1)$ 
    if level is null: break               # book ran dry
    if incoming is not MARKET and prices do not cross: break
    resting = level.head                  # oldest order first (FIFO)
    qty = min(incoming.remaining, resting.remaining)
    fill both orders by qty; reduce level aggregates
    emit TradeEvent at resting price (maker = resting, taker = incoming)
    if resting fully filled: unlink it; remove level if empty
    after any fill: scan pending_stops, trigger crossed stop orders

```

Fairness falls out of the structure rather than being checked explicitly: price priority comes from walking the sorted map from the best level, and time priority comes from always filling `level->head`. Partial fills are first-class - every order tracks original versus filled quantity and moves through the status state machine in Figure 3, whose transitions the test suite asserts directly.

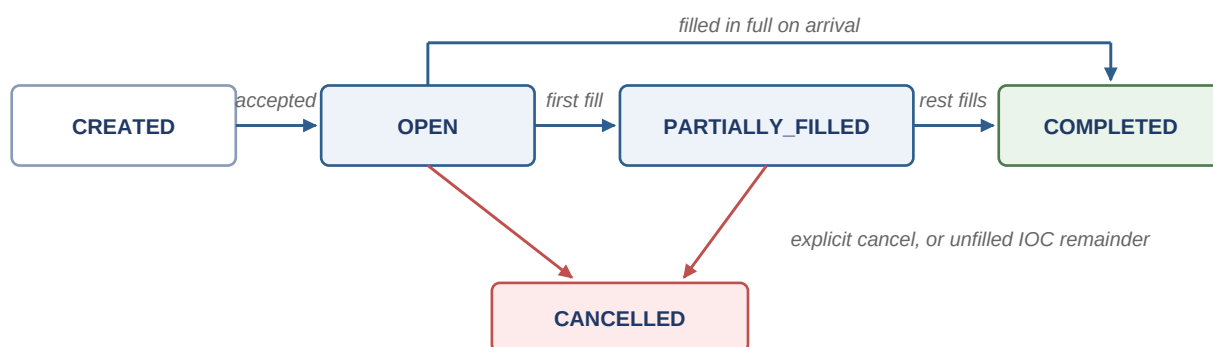


Figure 3 - Order status lifecycle. A resting order opens on the book, fills in one or more steps, and ends either fully executed or cancelled.

4.2 Order types

Type	Executes	Rests on book	Unfilled remainder
LIMIT	if it crosses	yes	rests at limit price
MARKET	always, sweeps best prices	never	dropped if book runs dry
IOC	if it crosses	never	cancelled immediately
FOK	only if fully fillable	never	rejected whole (all-or-nothing)
STOP_LOSS	as MARKET once triggered	never	dropped if book runs dry
STOP_LIMIT	as LIMIT once triggered	after trigger	rests at limit price

Table 2 - Order type semantics.

4.3 Fill-or-Kill without rollback

FOK promises all-or-nothing execution. Instead of optimistically matching and rolling back on failure - which would require a transaction log over book state - the engine runs a read-only pre-scan (`can_fully_fill()`) that sums available quantity at crossing levels. Only if the order can fill completely does matching begin. The cost is up to 2x traversal on success; the benefit is that a failed FOK is **correct by construction** and provably leaves the book untouched, which the test suite asserts directly.

4.4 Stop orders

STOP_LOSS and STOP_LIMIT orders do not enter the book on arrival; they wait in a `pending_stops` collection. After every fill, the engine compares the last trade price against each pending stop: a BUY stop triggers when the last trade price rises to or above its stop price, a SELL stop when it falls to or below. A triggered STOP_LOSS re-enters the engine as a MARKET order and a STOP_LIMIT as a LIMIT order, so triggered orders reuse the same matching path as everything else. Because triggered stops can themselves trade and move the last price, stops can cascade; multiple stops at the same trigger price fire in FIFO insertion order.

5. Concurrency Model

The EventQueue is a classic producer-consumer queue: `push()` appends under a `std::mutex` and signals a condition variable; `pop()` blocks in `cv.wait(lock, [&]{ return !q.empty(); })`, with the predicate guarding against spurious wakeups. The worker exits cleanly on a STOP event, so shutdown is an ordinary message rather than a special mechanism.

What makes this design work is that the queue is the **only** synchronization point in the system. Exactly one thread consumes events, and that same thread owns the OrderBook, so no lock is ever taken during matching - data races are prevented by ownership, not by locking discipline. That is far easier to reason about and to test than fine-grained locking. The accepted limit is that one book runs on one core; the way to scale is per-symbol sharding, where each symbol gets its own queue and worker and

symbols scale out independently.

6. Supporting Subsystems

6.1 Volume-tiered maker-taker fees

Each fill charges the **taker** (the incoming order) and credits or charges the **maker** (the resting order that provided liquidity). The FeeCalculator tracks cumulative traded notional per user and promotes users monotonically through three tiers; upper tiers earn maker rebates (negative fees), mirroring how real venues incentivize liquidity provision. Self-matches (same user on both sides) are excluded from volume accumulation.

Tier	Cumulative notional	Maker fee	Taker fee
T0	≥ 0	0.00%	0.05%
T1	$\geq 100,000$	-0.01% (rebate)	0.04%
T2	$\geq 1,000,000$	-0.02% (rebate)	0.03%

Table 3 - Fee schedule.

6.2 Market data

Two views are exposed. The **BBO** (best bid and offer) reads the cached top-of-book pointers in $O(1)$. The **L2 snapshot** returns price and aggregate quantity for the top D levels per side in $O(D)$, iterating the sorted maps from the best price. Both have stream-printing helpers for human-readable inspection.

6.3 Trade publication

The engine emits each fill as a **TradeEvent** (buy/sell order IDs, price, quantity, both fees, engine and wall-clock timestamps) through an abstract **TradePublisher** interface. The engine depends only on the interface - the in-memory implementation used in tests could be swapped for a file logger or network stream without touching matching code (dependency inversion / strategy pattern).

6.4 Timestamps

Two clocks serve two purposes: a monotonic `steady_clock` drives event ordering, hardened so consecutive reads are strictly increasing even if the clock does not advance between calls, while `system_clock` provides wall-clock UTC for audit records, formatted as ISO-8601 with microsecond precision.

7. Testing and Verification

Testing is invariant-driven: rather than only checking outputs, tests assert the structural properties the design promises (no empty price levels, FIFO within a level, BBO matches the real top-of-book, a failed FOK changes nothing). Each scenario runs against a fresh fixture wiring FeeCalculator → OrderBook → MatchingEngine → publisher. The suite totals **165 assertions across 15 scenarios**, all passing, capped by a 13-part combined test that exercises every subsystem in one session.

Scenario	What it proves
Limit orders	resting, multi-level sweeps, FIFO within a price level
Market orders	sweeps liquidity, partial fill, never rests on the book
IOC / FOK	remainder auto-cancel; all-or-nothing with no side effects on failure
Status machine	CREATED, OPEN, PARTIALLY_FILLED, COMPLETED, CANCELLED transitions
Cancellation	$O(1)$ cancel after partial fill; double-cancel is safe
Global invariants	a fully matched book empties both sides

Scenario	What it proves
Fees	tier promotion at notional thresholds; maker rebates applied
Market data	BBO and L2 depth reflect true book state
Trade stream	publisher receives every fill with fees and timestamps
Stop orders	trigger rules, FIFO triggering, cascading stops
Timestamps	monotonic, unique, ISO-8601 formatting
Event queue (async)	engine on a real second thread; blocking pop; clean STOP shutdown

Table 4 - Test coverage by scenario.

The asynchronous test deserves note: concurrent tests assert only properties that hold for **any** valid thread interleaving (totals, final state, event counts), never exact orderings, which keeps them deterministic and free of flakiness.

8. Challenges Faced and Trade-offs

Each subsection states the problem, the decision taken, and what was knowingly given up.

8.1 Making cancellation $O(1)$

A naive book stores orders in containers that require searching a price level to cancel - $O(N)$ in the level size. The fix is the combination of the hash index (find the node) and the intrusive list (unlink it in place). The cost is manual pointer discipline: every insert, fill, and cancel must keep next/prev pointers, level aggregates, and the index consistent. Copy construction on Order and PriceLevel is explicitly deleted so a node can never be accidentally duplicated while linked.

8.2 Keeping the cached BBO truthful

Caching best_bid/best_ask makes reads $O(1)$ but creates a consistency obligation: the moment a fill or cancel empties the best level, the cache must move to the next level (or null) before anything else observes it. Getting every structural mutation path to refresh the cache was the most persistent source of subtle bugs while building the engine - which is exactly why the market data tests assert BBO correctness after every kind of mutation.

8.3 FOK: pre-scan versus rollback

The tempting implementation - match optimistically, undo on failure - quietly turns the matching loop into a transactional system. I went with a read-only pre-scan instead: it costs roughly double traversal on successful FOKs, but it eliminates an entire class of partial-undo bugs. Correctness won over the constant factor.

8.4 Stop-order triggering at scale

Stops must be evaluated against the last trade price, which can change on every fill. The current implementation scans all pending stops after each fill - $O(S)$, the engine's primary known bottleneck. The documented fix is a sorted index (`std::multimap` keyed by trigger price), reducing the check to $O(\log S + T)$ for T triggered orders; it is a drop-in replacement since trigger semantics are already isolated. A second subtlety is cascading: a triggered stop can trade and trigger further stops, so triggering is re-entrant by design.

8.5 Memory ownership with raw pointers

The book links nodes with raw pointers rather than smart pointers: `shared_ptr` would add atomic reference-count traffic to the hot path, and intrusive links inherently express non-owning relationships. Safety instead comes from a strict single-owner rule (the OrderBook owns resting state) plus the single-writer thread model, which removes concurrent use-after-free by construction. The accepted risk

is that an exception between allocation and linkage could leak - acceptable in a simulation, and the roadmap addresses it with memory pooling.

8.6 Concurrency without locking the book

I initially considered locking the book so multiple threads could match concurrently, then realized that matching is inherently serial per symbol - every order may touch the same best level. That observation led to the single-writer design: accept one core per book, eliminate every lock past the queue, and scale by sharding symbols across workers. Throughput per book is bounded, but correctness reasoning collapses from "all possible interleavings" to "one sequential program".

8.7 Floating-point prices and unbounded history

Two pragmatic shortcuts are acknowledged rather than hidden. Prices use `double`, which is convenient for simulation but risks precision drift in comparisons; the production-grade fix is fixed-point integer ticks (`int64_t`). And the engine retains all trades in memory as an audit trail, which grows without bound in long sessions; the fix is streaming through the `TradePublisher` with a bounded ring buffer.

9. Limitations and Roadmap

Known limitations follow directly from the scoping decisions above: single symbol, in-memory only, double-precision prices, linear stop scan, and no network interface. Planned enhancements, in rough priority order:

- **Multi-symbol routing** - a symbol-to-`OrderBook` hash map with one worker per symbol (embarrassingly parallel scale-out).
- **Sorted stop index** - $O(\log S + T)$ trigger lookup via a multimap keyed by stop price.
- **Fixed-point pricing** - `int64_t` ticks to eliminate floating-point comparison risk.
- **Memory pooling** - object pools for `Order` and `PriceLevel` to remove allocation from the hot path and close leak windows.
- **StopOrderManager** - decouple stop bookkeeping from the `OrderBook`, removing the current direct coupling to `pending_stops`.
- **Trade streaming** - bounded ring buffer plus external publication instead of unbounded in-memory history.
- **Cache-friendly book** - replace the red-black tree with a flat structure if profiling shows pointer-chasing dominating.

Conclusion

TradeForge delivers what it set out to build: a matching engine whose fairness guarantees come from its data structures, whose thread safety comes from ownership rather than locks, and whose claims are backed by 165 passing assertions. Just as deliberate is what it does not claim - the known bottlenecks are written down with concrete fixes, and each future enhancement slots into an interface that already exists. The result is a compact, defensible foundation for exchange-system experimentation.